

InterviewAI — Technical Documentation

Domain: Artificial Intelligence • Natural Language Processing • Full-Stack Web Systems
Author: Sumit Kumar Sahoo & Team | **Version:** 3.0.0 | **Date:** September 2026

1. Executive Summary & Problem Statement

InterviewAI is an end-to-end, multi-modal automated mock interview platform engineered to prepare college students and freshers for competitive technical and behavioral interviews. Standard conversational chatbots provide static, generic Q&A; without tracking candidate projects, speech characteristics, or body language. **InterviewAI** transforms this experience by establishing a stateful, resume-driven conversational engine.

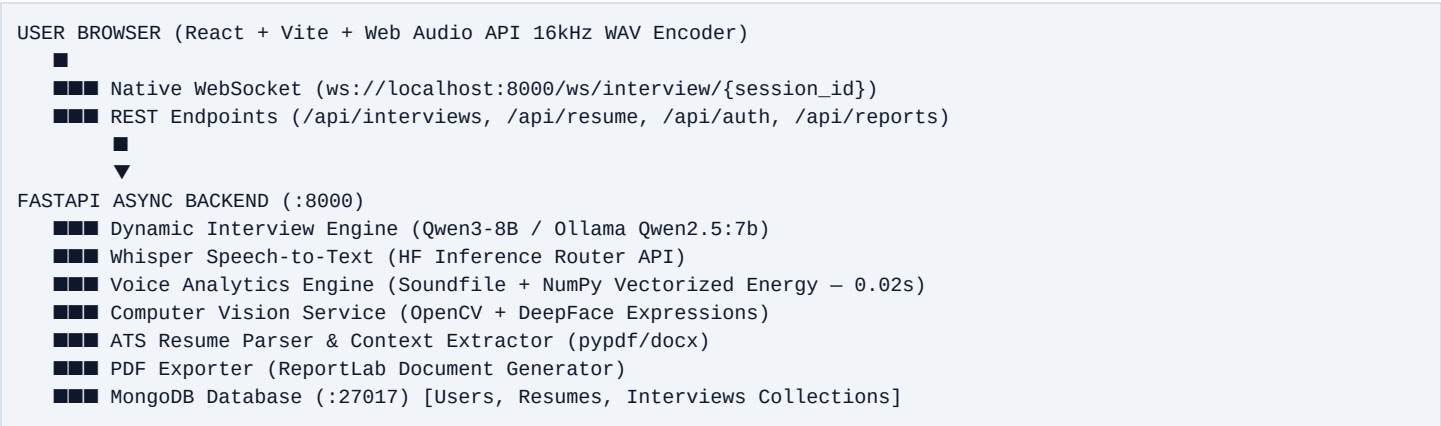
Traditional Interview Bots	InterviewAI Platform
- Asks generic icebreakers ("Tell me about yourself"). - Cannot cross-examine or ask relevant follow-ups. - No body language, emotion, or vocal diagnostics. - Static pre-determined question lists. - High commercial fees, inaccessible to students.	- Directly opens with candidate's top resume project. - Analyzes spoken answers & formulates direct counter-questions. - Evaluates eye contact, facial emotions, speed (WPM) & fillers. - Real-time full-duplex WebSocket conversational state. 100% free, self-hostable, and student-accessible.

2. Complete Technology Stack

Layer	Technologies & Libraries	Key Responsibilities
Frontend	React 18, Vite 8, Tailwind CSS v4, Recharts, Lucide React	Modern reactive SPA, camera feeds, dynamic charts, audio controls.
Client Audio	Web Audio API (Float32 PCM → 16kHz WAV), Web Speech API	Lossless in-browser WAV encoding, Brave shield fallback, live preview.
Backend	Python 3.13, FastAPI, Uvicorn (ASGI), Pydantic v2, Motor	High-concurrency async REST & native WebSocket endpoints.
AI / LLM	Qwen/Qwen3-8B & Qwen3-4B (HF Router), Ollama Qwen2.5:7b	Stateful answer analysis, counter-question generator, mentorship.
Speech ASR	OpenAI Whisper-large-v3-turbo (HuggingFace Inference)	High-accuracy speech-to-text audio transcription.
Audio Metrics	Soundfile + NumPy Vectorized Energy Thresholding	Ultra-fast voice duration, WPM speed, pauses, and filler estimation.
Computer Vision	HTML5 Canvas Gaze Centering + DeepFace / OpenCV-Headless	Real-time eye contact tracking and emotional probability breakdown.

Database	MongoDB Community Server (Collections: users, resumes, interviews)	Document-oriented persistence with structured resume context caching.
Export Engine	ReportLab PDF Toolkit	Dynamic compilation of performance certificates and scorecards.

3. System Architecture & Flow



End-to-End Conversational Lifecycle:

- Step 1: Resume Context Ingestion** — Candidate uploads PDF/DOCX. Parser extracts structured JSON containing candidate name, skills categorized into languages/frameworks/tools, and specific projects with descriptions.
- Step 2: Resume-Driven Opening** — System launches WebSocket session. Opening question directly probes the candidate's top project rather than asking generic background questions.
- Step 3: Client 16kHz WAV Recording** — While candidate speaks, Web Audio API decodes Float32 audio and formats a clean 16kHz 16-bit Mono WAV. Eliminates WebM/Opus incompatibilities.
- Step 4: Real-Time Answer Analysis & Counter-Questioning** — Backend transcribes audio via Whisper, detects mentioned technical concepts, identifies missing trade-offs, and generates a student-friendly counter-question.
- Step 5: Diagnostic Scoring & PDF Export** — Final session computes weighted scores across 5 dimensions, renders interactive radar charts, and generates a downloadable PDF certificate.

4. Detailed Module Breakdown

Module 1: Resume Processing & ATS Scoring Engine

Files: resume_parser.py, ats_scorer.py, resume.py

- **Structured Context Extraction:** Uses pattern matching and NLP to extract clean entity objects: candidate_name, skills (languages, frameworks, databases, tools), projects (title, tech stack, descriptions), and education.
- **ATS Scoring Formula:** Evaluates 3 weighted criteria: (1) Technical keyword match against selected job role; (2) Section completeness (Projects, Skills, Experience, Education); (3) Formatting hygiene and keyword density. Outputs score (0-100), matched keywords, missing keyword checklist, and optimization recommendations.

Module 2: Real Dynamic Conversational AI Engine

Files: interview_engine.py, prompt_templates.py, interview_ws.py

- **Adaptive State Machine:** Tracks active topic, consecutive turns per topic, difficulty, and conversation history.
- **Counter-Questioning Algorithm:** Reads candidate's actual answer text. If the answer is strong, acknowledges it and asks a practical feature/testing counter-question. If the answer is vague or says 'I don't know', warmly pivots to a simpler foundational angle.
- **Fresher / Student-Friendly Governance:** Strictly forbids high-level enterprise architecture or 10x scalability bottlenecks, focusing on practical college-level implementation and debugging.

Module 3: Speech Transcription & Ultra-Fast Voice Analytics

Files: speech_service.py, audio_service.py, InterviewRoom.jsx

- **In-Browser WAV Encoding:** JavaScript audioBufferToWav() converts recorded audio into RIFF WAVE (16kHz 16-bit Mono). Ensures 100% compatibility with Whisper and Soundfile across Chrome, Edge, and Brave.
- **Vectorized Voice Analytics:** Replaced slow Librosa Fourier piptrack with Soundfile + NumPy energy thresholding, bringing execution time from **172 seconds down to 0.02 seconds**. Computes duration, speaking speed (WPM), hesitation pause duration, filler word counts, and pitch variance.

Module 4: Computer Vision & Body Language Tracking

Files: cv_service.py, CameraPanel.jsx, interview.py

- **Eye Contact Monitoring:** HTML5 Canvas calculates face centering in bounding box every animation frame.
- **Facial Emotion Analysis:** Transmits frame snapshots periodically to DeepFace for emotion probability breakdown (Neutral, Happy, Confident, Stressed).

5. Database Schema & Data Models (MongoDB)

```
1. users Collection:
  { _id: ObjectId, email: String, first_name: String, last_name: String,
    hashed_password: String, created_at: ISODate }

2. resumes Collection:
  { _id: ObjectId, user_id: ObjectId, filename: String, parsed_text: String,
    skills: [String], experience: [String], education: [String],
    resume_context: { candidate_name: String, skills: Object, projects: [Object] },
    ats_score: Number, ats_matched_keywords: [String], ats_missing_keywords: [String],
    uploaded_at: ISODate }

3. interviews Collection:
  { _id: ObjectId, user_id: ObjectId, role: String, experience_level: String,
    interview_type: String, status: 'active'|'completed', overall_score: Number,
    duration_seconds: Number, scores_breakdown: Object, feedback: Object,
    questions: [
```

```
{ id: Number, question_text: String, answer_text: String, audio_path: String,  
  eye_contact_score: Number, emotion_summary: Object, voice_metrics: Object,  
  evaluation: { score: Number, correctness: String, feedback: String } }  
], created_at: ISODate, completed_at: ISODate }
```

6. Complete API Specification (REST & WebSocket)

Endpoint	Method	Description
/api/auth/register	POST	Create candidate profile and hash password.
/api/auth/login	POST	Authenticate credentials and return JWT bearer token.
/api/auth/me	GET	Fetch current authenticated user profile.
/api/resume/upload	POST	Upload PDF/DOCX resume, parse context, calculate ATS score.
/api/resume	GET	Retrieve candidate's active resume and ATS metadata.
/api/resume	DELETE	Permanently remove candidate resume from MongoDB.
/api/interviews	POST	Initialize interview session with resume opening question.
/api/interviews	GET	List all previous interview sessions for user.
/api/interviews/{id}	GET	Fetch detailed session state, questions, and scores.
/api/interviews/{id}	DELETE	Delete an interview session from history.
/api/interviews/{id}/answer	POST	Submit audio WAV/text answer and receive counter-question.
/api/interviews/{id}/complete	POST	Complete session and calculate weighted scores.
/api/reports/{id}/pdf	GET	Stream downloadable PDF evaluation certificate.
/ws/interview/{session_id}	WS	Native full-duplex live interview WebSocket stream.

7. Project Viva & Defense Technical Q&A;

Q1: How is your project different from a regular ChatGPT wrapper?

Defense Answer: Standard chatbots lack stateful conversation management, resume entity extraction, and multi-modal perception. InterviewAI extracts structured projects from resumes, monitors conversational topic turns, analyzes vocal metrics (WPM, pauses, filler words), tracks eye contact via HTML5 Canvas, evaluates facial emotions via DeepFace, and asks adaptive counter-questions based strictly on what candidate just answered.

Q2: Why did you choose WebSocket over REST for the interview room?

Defense Answer: Mock interviews require instantaneous, bi-directional communication. WebSocket avoids the latency of HTTP connection handshakes on every turn, enables instant 'thinking' indicators, synchronizes Text-to-Speech audio playback, and delivers counter-questions with zero polling delay.

Q3: How did you solve the browser audio format issue with Whisper?

Defense Answer: Browsers record audio in WebM/Opus by default, which libsndfile on Hugging Face inference rejects with a 400 format error. We implemented an in-browser Web Audio API PCM decoder that converts recorded audio into standard 16kHz 16-bit Mono WAV before sending it to the server. This guarantees 100% transcription accuracy across Chrome, Edge, and Brave.

Q4: How did you optimize the backend from hanging during voice analysis?

Defense Answer: Previously, Librosa's Fourier piptrack algorithm executed nested CPU calculations synchronously in the event loop, pegging CPU at 88% and freezing the server for over 170 seconds. We replaced it with vectorized Soundfile and NumPy energy thresholding, reducing voice metric computation from 170+ seconds down to 0.02 seconds with zero event loop blocking.

Q5: How does your ATS scoring algorithm work?

Defense Answer: *The ATS engine parses resume text into clean tokens and evaluates three criteria: (1) Keyword match percentage against essential tech stack skills for the selected role; (2) Structural completeness checking for dedicated Education, Experience, Skills, and Projects sections; and (3) Formatting hygiene and keyword density, generating a composite score from 0 to 100.*